



Microcontrollers

Rex Joseph

Department of Electrical and
Electronics Engineering

Recap: MSP430 Architecture

Looked at:

Memory map

Registers

Application examples

Addressing Modes



Processor – Architecture – Instruction set

- MPY ACC, loc16, #16bit
- MPY ACC, T, loc16
- MPY P, loc16, #16bit
- MPY P, T, loc16
- MPYA P, loc16, #16bit
- MPYA P, T, loc16
- MPYB ACC, T, #8bit
- MPYB P, T, #8bit
- MPYS P, T, loc16
- MPYU P, T, loc16
- MPYU ACC, T, loc16
- MPYXU ACC, T, loc16
- MPYXU P, T, loc16

Processor – Architecture – Instruction set

16x16 Multiply Operations

DMAC	ACC:P,loc32,*XAR7/++	16-bit dual multiply and accumulate
MAC	P,loc16,0:pma	Multiply and accumulate
MAC	P,loc16,*XAR7/++	Multiply and Accumulate
MPY	P,T,loc16	16 X 16 multiply
MPY	P,loc16,#16bit	16 X 16-bit multiply
MPY	ACC,T,loc16	16 X 16-bit multiply
MPY	ACC,loc16,#16bit	16 X 16-bit multiply
MPYA	P,loc16,#16bit	16 X 16-bit multiply and add previous product
MPYA	P,T,loc16	16 X 16-bit multiply and add previous product
MPYB	P,T,#8bit	Multiply signed value by unsigned 8-bit constant
MPYS	P,T,loc16	16 X 16-bit multiply and subtract

Processor – Architecture – Instruction set

32x32 Multiply Operations

IMACL	P,loc32,*XAR7/++	Signed 32 X 32-bit multiply and accumulate (lower half)
IMPYAL	P,XT,loc32	Signed 32-bit multiply (lower half) and add previous P
IMPYL	P,XT,loc32	Signed 32 X 32-bit multiply (lower half)
IMPYL	ACC,XT,loc32	Signed 32 X 32-bit multiply (lower half)
IMPYSL	P,XT,loc32	Signed 32-bit multiply (lower half) and subtract P
IMPYXUL	P,XT,loc32	Signed 32 X unsigned 32-bit multiply (lower half)
QMACL	P,loc32,*XAR7/++	Signed 32 X 32-bit multiply and accumulate (upper half)
QMPYAL	P,XT,loc32	Signed 32-bit multiply (upper half) and add previous P
QMPYL	ACC,XT,loc32	Signed 32 X 32-bit multiply (upper half)
QMPYL	P,XT,loc32	Signed 32 X 32-bit multiply (upper half)
QMPYSL	P,XT,loc32	Signed 32-bit multiply (upper half) and subtract previous P
QMPYUL	P,XT,loc32	Unsigned 32 X 32-bit multiply (upper half)
QMPYXUL	P,XT,loc32	Signed 32 X unsigned 32-bit multiply (upper half)

Processor – Architecture – Instruction set

MPYU P,T,loc16

Unsigned 16 X 16 Multiply

SYNTAX OPTIONS	OPCODE	OBJMODE	RPT	CYC
MPYU P,T,loc16	0011 0111 LLLL LLLL	X	–	1

Operands

P Product register

T Multiplicand register

loc16 Addressing mode (see Chapter 5)

Description

Multiply the signed 16-bit content of the T register by the signed 16-bit contents of the location pointed to by the “loc16” addressing mode and store the 32-bit result in the P register:

$P = \text{unsigned } T * \text{unsigned } [\text{loc16}];$

Flags and Modes

None

Repeat

This instruction is not repeatable. If this instruction follows the RPT instruction, it resets the repeat counter (RPTC) and executes only once.

Example

```
; Calculate unsigned value: Y32 = X16 * M16;
MOV    T,@X16          ; T = X16
MPYU   P,T,@M16         ; P = T * M16
MOVL   @Y,P             ; Store result into Y32
```

Processor – Architecture – Instruction set

MPYA P,loc16,#16bit		16 X 16-Bit Multiply and Add Previous Product			
SYNTAX OPTIONS		OPCODE	OBJMODE	RPT	CYC
MPYA P,loc16,#16bit		0001 0101 LLLL LLLL CCCC CCCC CCCC CCCC	X	–	1

Operands

P Product register

loc16 Addressing mode (see Chapter 5)

#16bit 16-bit immediate constant value

Description

Add the previous product (stored in the P register), shifted as specified by the product shift mode (PM) bits, to the ACC register. Load the T register with the content of the location pointed to by the "loc16" addressing mode. Multiply the signed 16-bit content of the T register by the signed 16-bit constant value and store the 32-bit result in the P register:

ACC = ACC + P << PM;
T = [loc16];
P = signed T * signed 16bit;

TMS320F2812

Flags and Modes

Z After the operation, the Z flag is set if the ACC is zero, else Z is cleared.

N After the addition, the N flag is set if bit 31 of the ACC is 1, else N is cleared.

C If the addition generates a carry, C is set; otherwise C is cleared.

V If an overflow occurs, V is set; otherwise V is not affected.

OVC If overflow mode is disabled; and if the operation generates a positive overflow, then the counter is incremented. If overflow mode is disabled; and if the operation generates a negative overflow, then the counter is decremented.

OVM If overflow mode bit is set; then the ACC value will saturate maximum positive (0x7FFFFFFF) or maximum negative (0x80000000) if the operation overflowed.

PM The value in the PM bits sets the shift mode for the output operation from the product register. If the product shift value is positive (logical left shift operation), then the low bits are zero filled. If the product shift value is negative (arithmetic right shift operation), the upper bits are sign extended.

Repeat

This instruction is not repeatable. If this instruction follows the RPT instruction, it resets the repeat counter (RPTC) and executes only once.

CISC!

- CISC is an acronym for Complex Instruction Set Computer and are chips that are easy to program and which make efficient use of memory.
- Since the earliest machines were programmed in assembly language and **memory was slow and expensive**, the CISC philosophy made sense
- Most common microprocessor designs such as the Intel 80x86 and Motorola 68K series followed the CISC philosophy.

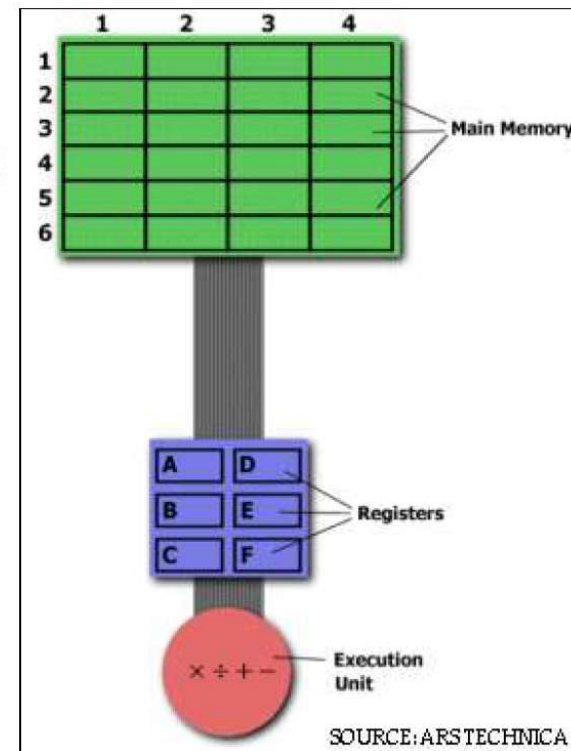


Multiply and store

product of two numbers - one stored in location 2:3 and another stored in location 5:2 - and then store the product back in the location 2:3.

CISC: MULT 2:3 , 5:2 → ($a=a*b$)

RISC: LOAD A , 2:3
 LOAD B , 5:2
 MUL A , B
 STORE A, 2:3



Instruction Set

Instruction Set

The complete MSP430 instruction set consists of 27 core instructions and 24 emulated instructions.

The **core instructions** are instructions that have **unique op-codes** decoded by the CPU.

The **emulated instructions** are instructions that make code easier to write and read, but **do not have op-codes themselves**, instead they are **replaced** automatically by the **assembler** with an **equivalent core instruction**.

There is no code or performance penalty for using emulated instruction.

There are three core instruction formats:

- Dual operand
- Single operand
- Jump

Instruction Set

Instruction Set

All single-operand and dual-operand instructions can be byte or word instructions by using .B or .W extensions.

Byte instructions are used to access byte data or byte peripherals.

Word instructions are used to access word data or word peripherals.
If **no extension** is used, the instruction is a **word instruction**.

Instruction Set

The source and destination of an instruction are defined by the following fields:

src	The source operand defined by As and S-reg
dst	The destination operand defined by Ad and D-reg
As	The addressing bits responsible for the addressing mode used for the source (src)
S-reg	The working register used for the source (src)
Ad	The addressing bits responsible for the addressing mode used for the destination (dst)
D-reg	The working register used for the destination (dst)

B/W Byte or word operation:

0: word operation

1: byte operation

Instruction Set

Destination Address

Destination addresses are valid anywhere in the memory map.

However, when using an instruction that modifies the contents of the destination, the user must ensure the destination address is writable.

For example, a **masked-ROM** location would be a **valid destination address**, but the contents are not modifiable, so the results of the instruction would be lost.

MSP430 Instruction Set

Double-Operand (Format I) Instructions

*	The status bit is affected
–	The status bit is not affected
0	The status bit is cleared
1	The status bit is set

The instructions CMP and SUB are identical except for the storage of the result.

The same is true for the BIT and AND instructions

12 instructions in all

MSP430 Instruction Set

Double-Operand (Format I) Instructions

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code				S-Reg				Ad	B/W	As	D-Reg				

Mnemonic	S-Reg, D-Reg	Operation	Status Bits			
			V	N	Z	C
MOV(.B)	src,dst	src → dst	-	-	-	-
ADD(.B)	src,dst	src + dst → dst	*	*	*	*
ADDC(.B)	src,dst	src + dst + C → dst	*	*	*	*
SUB(.B)	src,dst	dst + .not.src + 1 → dst	*	*	*	*
SUBC(.B)	src,dst	dst + .not.src + C → dst	*	*	*	*
CMP(.B)	src,dst	dst - src	*	*	*	*
DADD(.B)	src,dst	src + dst + C → dst (decimally)	*	*	*	*
BIT(.B)	src,dst	src .and. dst	0	*	*	*
BIC(.B)	src,dst	not.src .and. dst → dst	-	-	-	-
BIS(.B)	src,dst	src .or. dst → dst	-	-	-	-
XOR(.B)	src,dst	src .xor. dst → dst	*	*	*	*
AND(.B)	src,dst	src .and. dst → dst	0	*	*	*

MSP430 Instruction Set

Single-Operand (Format II) Instructions

*	The status bit is affected
–	The status bit is not affected
0	The status bit is cleared
1	The status bit is set

All addressing modes are possible for the CALL instruction.

If the symbolic mode (ADDRESS), the immediate mode (#N), the absolute mode (&EDE) or the indexed mode x(RN) is used, the word that follows contains the address information

7 instructions

MSP430 Instruction Set

Single-Operand (Format II) Instructions

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code									B/W	Ad		D/S-Reg			

Mnemonic	S-Reg, D-Reg	Operation	Status Bits			
			V	N	Z	C
RRC (.B)	dst	$C \rightarrow \text{MSB} \rightarrow \dots \rightarrow \text{LSB} \rightarrow C$	*	*	*	*
RRA (.B)	dst	$\text{MSB} \rightarrow \text{MSB} \rightarrow \dots \rightarrow \text{LSB} \rightarrow C$	0	*	*	*
PUSH (.B)	src	$\text{SP} - 2 \rightarrow \text{SP}, \text{src} \rightarrow @\text{SP}$	-	-	-	-
SWPB	dst	Swap bytes	-	-	-	-
CALL	dst	$\text{SP} - 2 \rightarrow \text{SP}, \text{PC} + 2 \rightarrow @\text{SP}$	-	-	-	-
		dst $\rightarrow \text{PC}$				
RETI		$\text{TOS} \rightarrow \text{SR}, \text{SP} + 2 \rightarrow \text{SP}$	*	*	*	*
		$\text{TOS} \rightarrow \text{PC}, \text{SP} + 2 \rightarrow \text{SP}$				
SXT	dst	Bit 7 $\rightarrow$ Bit 8.....Bit 15	0	*	*	*

MSP430 Instruction Set

Jumps

Conditional jumps support program branching relative to the PC and do not affect the status bits.

The possible jump range is from -511 to $+512$ words relative to the PC value at the jump instruction.

The 10-bit program counter offset is treated as a **signed 10-bit value** that is doubled and added to the program counter:

$$PC_{\text{new}} = PC_{\text{old}} + 2 + PC_{\text{offset}} \times 2$$

8 instructions

MSP430 Instruction Set

Jumps

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code			C			10-Bit PC Offset									

Mnemonic	S-Reg, D-Reg	Operation
JEQ/JZ	Label	Jump to label if zero bit is set
JNE/JNZ	Label	Jump to label if zero bit is reset
JC	Label	Jump to label if carry bit is set
JNC	Label	Jump to label if carry bit is reset
JN	Label	Jump to label if negative bit is set
JGE	Label	Jump to label if $(N \text{ .XOR. } V) = 0$
JL	Label	Jump to label if $(N \text{ .XOR. } V) = 1$
JMP	Label	Jump to label unconditionally

MSP430 Instruction Set

Instruction Cycles and Lengths

The number of CPU clock cycles required for an instruction depends on the instruction format and the addressing modes used - not the instruction itself.

The number of clock cycles refers to the MCLK.

We can compute execution time of code

MSP430 Instruction Set

Interrupt and Reset Cycles

Action	No. of Cycles	Length of Instruction
Return from interrupt (RETI)	5	1
Interrupt accepted	6	-
WDT reset	4	-
Reset ($\overline{\text{RST}}$ /NMI)	4	-

Format-III (Jump) Instruction Cycles and Lengths

All jump instructions require one code word, and take two CPU cycles to execute, regardless of whether the jump is taken or not.

MSP430 Instruction Set

Format-II (Single Operand) Instruction Cycles and Lengths

Addressing Mode	No. of Cycles			Length of Instruction	Example
	RRA, RRC SWPB, SXT	PUSH	CALL		
Rn	1	3	4	1	SWPB R5
@Rn	3	4	4	1	RRC @R9
@Rn+	3	5	5	1	SWPB @R10+
#N	(See note)	4	5	2	CALL #0F000h
X(Rn)	4	5	5	2	CALL 2 (R7)
EDE	4	5	5	2	PUSH EDE
&EDE	4	5	5	2	SXT &EDE

Instruction Format II Immédiate Mode

Do not use instruction RRA , RRC , SWPB , and SXT with the immediate mode in the destination field.

Use of these in the immediate mode results in an unpredictable program operation

MSP430 Instruction Set

Complete Instruction Set

(1) Emulated Instruction



Mnemonic		Description		V	N	Z	C
ADC (.B) ⁽¹⁾	dst	Add C to destination	dst + C → dst	*	*	*	*
ADD (.B)	src, dst	Add source to destination	src + dst → dst	*	*	*	*
ADDC (.B)	src, dst	Add source and C to destination	src + dst + C → dst	*	*	*	*
AND (.B)	src, dst	AND source and destination	src .and. dst → dst	0	*	*	*
BIC (.B)	src, dst	Clear bits in destination	not.src .and. dst → dst	-	-	-	-
BIS (.B)	src, dst	Set bits in destination	src .or. dst → dst	-	-	-	-
BIT (.B)	src, dst	Test bits in destination	src .and. dst	0	*	*	*
BR ⁽¹⁾	dst	Branch to destination	dst → PC	-	-	-	-
CALL	dst	Call destination	PC+2 → stack, dst → PC	-	-	-	-
CLR (.B) ⁽¹⁾	dst	Clear destination	0 → dst	-	-	-	-
CLRC ⁽¹⁾		Clear C	0 → C	-	-	-	0
CLRN ⁽¹⁾		Clear N	0 → N	-	0	-	-
CLRZ ⁽¹⁾		Clear Z	0 → Z	-	-	0	-
CMP (.B)	src, dst	Compare source and destination	dst - src	*	*	*	*

MSP430 Instruction Set

Complete Instruction Set



Mnemonic		Description		V	N	Z	C
DADC (.B) ⁽¹⁾	dst	Add C decimally to destination	$\text{dst} + \text{C} \rightarrow \text{dst}$ (decimally)	*	*	*	*
DADD (.B)	src, dst	Add source and C decimally to dst	$\text{src} + \text{dst} + \text{C} \rightarrow \text{dst}$ (decimally)	*	*	*	*
DEC (.B) ⁽¹⁾	dst	Decrement destination	$\text{dst} - 1 \rightarrow \text{dst}$	*	*	*	*
DECD (.B) ⁽¹⁾	dst	Double-decrement destination	$\text{dst} - 2 \rightarrow \text{dst}$	*	*	*	*
DINT ⁽¹⁾		Disable interrupts	$0 \rightarrow \text{GIE}$	-	-	-	-
EINT ⁽¹⁾		Enable interrupts	$1 \rightarrow \text{GIE}$	-	-	-	-

MSP430 Instruction Set

Complete Instruction Set



Mnemonic		Description		V	N	Z	C
INC (.B) ⁽¹⁾	dst	Increment destination	dst + 1 → dst	*	*	*	*
INCD (.B) ⁽¹⁾	dst	Double-increment destination	dst+2 → dst	*	*	*	*
INV (.B) ⁽¹⁾	dst	Invert destination	.not.dst → dst	*	*	*	*
JC/JHS	label	Jump if C set/Jump if higher or same		-	-	-	-
JEQ/JZ	label	Jump if equal/Jump if Z set		-	-	-	-
JGE	label	Jump if greater or equal		-	-	-	-
JL	label	Jump if less		-	-	-	-
JMP	label	Jump	PC + 2 × offset → PC	-	-	-	-
JN	label	Jump if N set		-	-	-	-
JNC/JLO	label	Jump if C not set/Jump if lower		-	-	-	-
JNE/JNZ	label	Jump if not equal/Jump if Z not set		-	-	-	-
MOV (.B)	src, dst	Move source to destination	src → dst	-	-	-	-
NOP ⁽¹⁾		No operation		-	-	-	-

MSP430 Instruction Set

Complete Instruction Set



Mnemonic		Description		V	N	Z	C
POP (.B) ⁽¹⁾	dst	Pop item from stack to destination	@SP → dst, SP+2 → SP	-	-	-	-
PUSH (.B)	src	Push source onto stack	SP - 2 → SP, src → @SP	-	-	-	-
RET ⁽¹⁾		Return from subroutine	@SP → PC, SP + 2 → SP	-	-	-	-
RETI		Return from interrupt		*	*	*	*
RLA (.B) ⁽¹⁾	dst	Rotate left arithmetically		*	*	*	*
RLC (.B) ⁽¹⁾	dst	Rotate left through C		*	*	*	*
RRA (.B)	dst	Rotate right arithmetically		0	*	*	*
RRC (.B)	dst	Rotate right through C		*	*	*	*

MSP430 Instruction Set

Complete Instruction Set



Mnemonic		Description		V	N	Z	C
SBC (.B) ⁽¹⁾	dst	Subtract not(C) from destination	$dst + 0FFFFh + C \rightarrow dst$	*	*	*	*
SETC ⁽¹⁾		Set C	$1 \rightarrow C$	-	-	-	1
SETN ⁽¹⁾		Set N	$1 \rightarrow N$	-	1	-	-
SETZ ⁽¹⁾		Set Z	$1 \rightarrow Z$	-	-	1	-
SUB (.B)	src, dst	Subtract source from destination	$dst + .not.src + 1 \rightarrow dst$	*	*	*	*
SUBC (.B)	src, dst	Subtract source and not(C) from dst	$dst + .not.src + C \rightarrow dst$	*	*	*	*
SWPB	dst	Swap bytes		-	-	-	-
SXT	dst	Extend sign		0	*	*	*
TST (.B) ⁽¹⁾	dst	Test destination	$dst + 0FFFFh + 1$	0	*	*	1
XOR (.B)	src, dst	Exclusive OR source and destination	$src .xor. dst \rightarrow dst$	*	*	*	*

System Resets and Interrupts

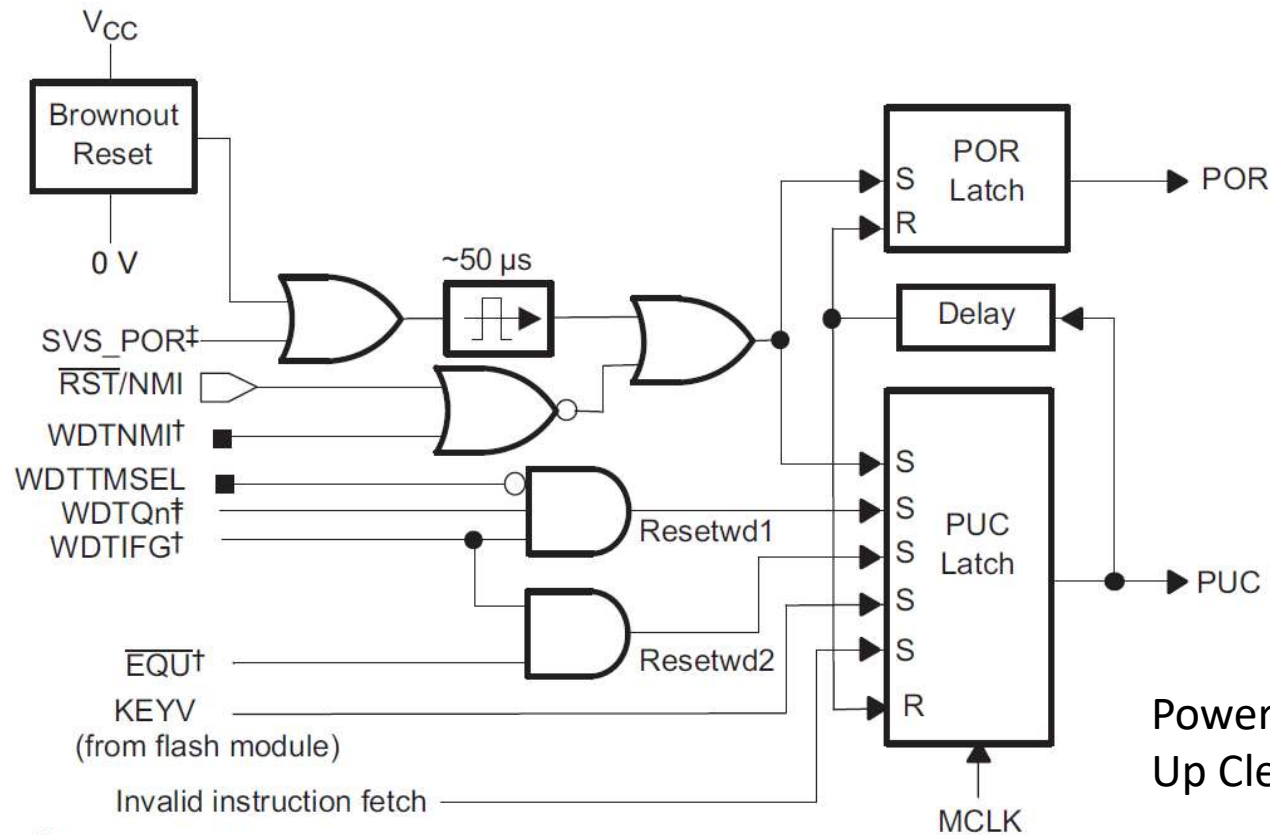
System Reset and Initialization

The system reset circuitry shown sources both a power-on reset (POR) and a power-up clear (PUC) signal.

Different events trigger these reset signals and different initial conditions exist depending on which signal was generated

Hardware circuit, and includes the brownout circuit

System Resets and Interrupts



Power-On Reset and Power-Up Clear Schematic

[†] From watchdog timer peripheral module

[‡] Devices with SVS only

System Resets and Interrupts

A POR is a device reset. A POR is only generated by the following three events:

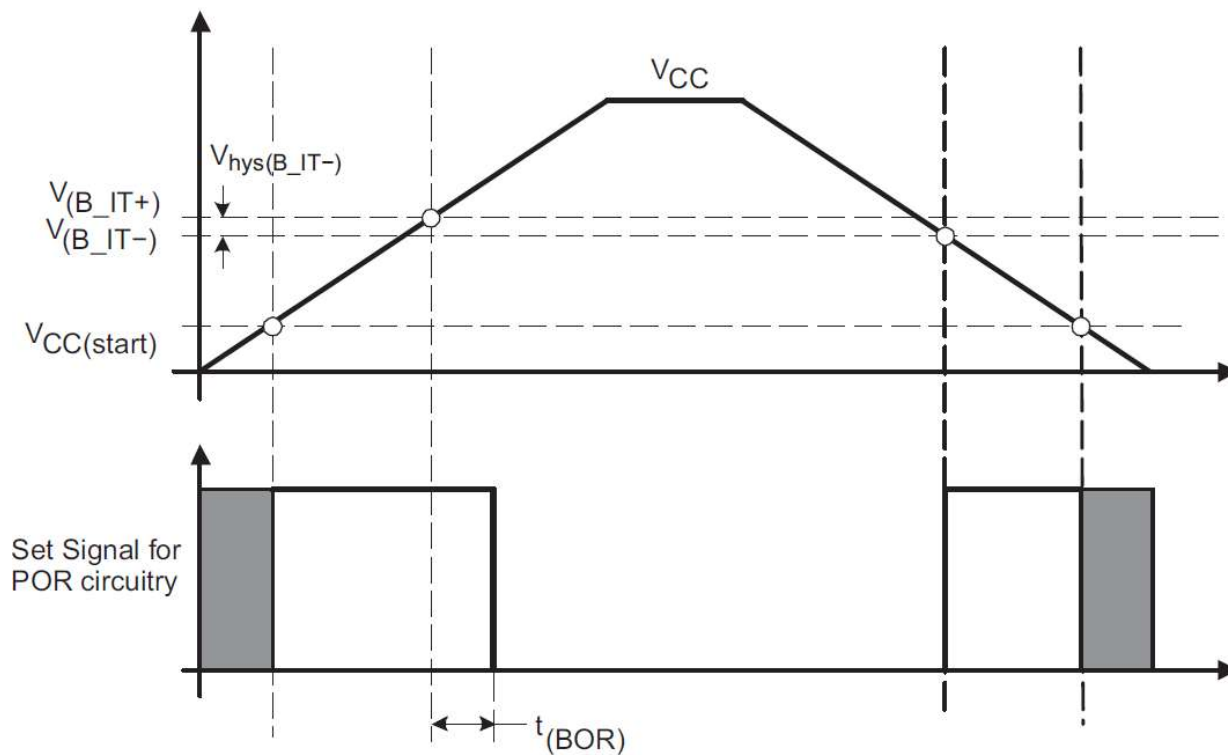
- Powering up the device
- A low signal on the RST/NMI pin when configured in the reset mode
- An SVS low condition when PORON = 1.

A PUC is always generated when a POR is generated, but a POR is not generated by a PUC. The following events trigger a PUC:

- A POR signal
- Watchdog timer expiration when in watchdog mode only
- Watchdog timer security key violation
- A Flash memory security key violation
- A CPU instruction fetch from the peripheral address range 0h to 01FFh

System Resets and Interrupts

Brownout Reset (BOR)



Brownout Timing
and Voltage levels

System Resets and Interrupts

Brownout Reset (BOR)

The brownout reset circuit detects low supply voltages such as when a supply voltage is applied to or removed from the VCC terminal.

The brownout reset circuit resets the device by triggering a POR signal when power is applied or removed.

The POR signal becomes active when VCC crosses the VCC(start) level.

It remains active until VCC crosses the V(B_IT+) threshold and the delay t(BOR) elapses.

The delay t(BOR) is adaptive being longer for a slow ramping VCC.

The hysteresis V_{hys}(B_ IT-) ensures that the supply voltage must drop below V(B_IT-) to generate another POR signal from the brownout reset circuitry.

As the V(B_IT-) level is significantly above the V_{min} level of the POR circuit, the BOR provides a reset for power failures where VCC does not fall below V_{min}.

System Resets and Interrupts

Device Initial Conditions After System Reset

After a POR, the initial MSP430 conditions are:

- The RST/NMI pin is configured in the reset mode.
- I/O pins are switched to **input mode**
- Other peripheral modules and registers are initialized as described
- Status register (SR) is **reset**.
- The watchdog timer powers up active in watchdog mode.
- Program counter (PC) is loaded with address contained at **reset vector** location (**0FFFFh**).
- If the reset vectors content is **0FFFFh** the device will be **disabled** for minimum power consumption.

System Resets and Interrupts

Software Initialization

After a system reset, **user software must initialize the MSP430** for the application requirements.

The following must occur:

- Initialize the SP, typically to the top of RAM.
- Initialize the watchdog to the requirements of the application.
- Configure peripheral modules to the requirements of the application.

Additionally, the watchdog timer, oscillator fault, and flash memory flags can be evaluated to determine the source of the reset.

System Resets and Interrupts

Interrupts

The interrupt priorities are fixed and defined by the arrangement of the modules in the connection chain.

The nearer a module is to the CPU/NMIRS, the higher the priority.

Interrupt priorities determine what interrupt is taken when **more than one interrupt is pending simultaneously**.

There are three types of interrupts:

- System reset
- (Non)-maskable NMI
- Maskable



THANK YOU

Rex Joseph

Department of Electrical and Electronics Engineering

rexjoseph@pes.edu

+91 80 6666 3333 Extn 515